

Gradient Descent and a NumPy Multilayer Perceptron (Lab 4-1)

CS 53744 Machine Learning Project

Due Date : 11:59 PM, October 7, 2026

Instructor: Prof. Jongmin Lee

How to use this note:

- Read it before the lab session. The lab will not repeat this material.
- Covered: the loss and its gradient, the forward pass of a network with a hidden layer, the chain rule and backpropagation, checking gradients with finite differences, learning rates and mini-batches. The emphasis is on what each method assumes and where it breaks.
- Not covered: code and function calls. See the docstrings in the skeleton `src/lab04_1.py`.
- Rules for this lab (environment, tasks, submission) are in the `README.md` in this folder. Course-wide policy (grading and AI use) was given in the policy slides of the first class and is not repeated here.

1 Learning objectives

1. Compute a forward pass with the sigmoid and binary cross-entropy, and implement it so the exponential never overflows. (Task 1)
2. Derive the partial derivative of the loss with respect to every parameter with the chain rule, and write it as matrix operations. (Task 2)
3. Check the analytic gradient against central finite differences using a relative error. (Task 2)
4. Implement the mini-batch gradient-descent update and the per-epoch history. (Task 3)
5. Design a controlled experiment in which only the learning rate varies, and read the three regimes off the result. (Task 4)

2 Background: how do you know your gradient is right?

In Lab 2, scikit-learn did the training for you. You called `fit`, a trained model came back, and you never had to look inside. In Lab 4-1 you write what is inside that `fit`. You derive the derivatives yourself and put them into code.

The data is 800 quality-control records from a heat press. There are only two settings, temperature and pressure, and a part passes only when *both* sit inside their acceptance window: temperature between 160 and 190 degrees, pressure between 35 and 65 kilopascals. The settings that pass therefore form a rectangle in the plane, and no straight line separates the inside of a rectangle from the outside. The logistic regression of Lab 2 cannot get much past the majority class here, so this data needs a hidden layer.

Training, in this note, means moving 33 numbers in the direction that reduces the loss, and the derivative supplies that direction. One wrong sign between the derivation and the code still reduces the loss, so when the loss drops a little and then stops, nothing

tells you whether the learning rate was too small or the gradient was wrong. That is why this lab has a checking step. You compare the gradient against finite differences and confirm the relative error is below 10^{-6} , and only then start varying the learning rate. Grading follows the same order: not how high the accuracy is, but whether you worked in it.

The four steps of Lab 4-1:

- **Forward pass:** compute the sigmoid, the hidden layer, the output, and the loss, keeping the intermediates. (Task 1)
- **Gradients and the check:** derive the gradients with the chain rule and verify them numerically. (Task 2)
- **Training loop:** update on mini-batches and record the loss each epoch. (Task 3)
- **Controlled comparison:** vary only the learning rate and hold everything else fixed. (Task 4)

3 Key concepts

3.1 One step in the direction that lowers the loss

Gradient descent moves the parameters a little way against the direction given by the derivative of the loss. With the parameters written θ and the learning rate η , the update is one line, and the loss is binary cross-entropy.

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}, \quad L = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log a_2^{(i)} + (1 - y_i) \log (1 - a_2^{(i)}) \right]$$

- A row with $y = 1$ keeps only $-\log a_2$, a row with $y = 0$ only $-\log(1 - a_2)$. Being confidently wrong grows the value fast. Dividing by n makes the loss an average that does not depend on the batch size.
- The gradient gives only a direction. How far to go is set by η (Figure 1).

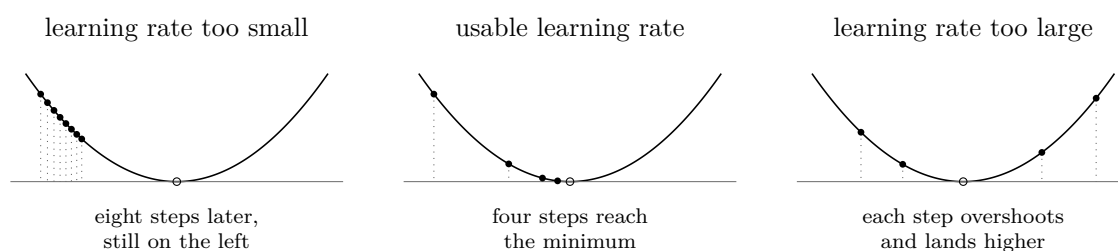


Figure 1. One gradient, three step lengths. The horizontal axis is one parameter, the vertical axis the loss, and the small circle marks the minimum. Each dot is one update, and the spacing of the dotted drop lines along the axis is the step length.

Assumptions and limits:

- Figure 1 has one parameter and a convex loss. The loss of the real network is not convex in 33 dimensions, so where descent settles depends on the starting weights and the learning rate. That is why `init_params` fixes the seed.
- Near a minimum the gradient shrinks, so the steps shorten on their own. That is why the loss curve flattens without anyone reducing the learning rate.

3.2 The forward pass of a network with a hidden layer

A multilayer perceptron alternates linear transformations with a nonlinear function. The architecture here is fixed at 2 inputs, 8 hidden units, and 1 output, so there are 24 weights and 9 biases, 33 numbers in all. The forward pass runs in one direction, from the input to the loss.

$$z_1 = XW^{(1)} + b^{(1)}, \quad a_1 = \sigma(z_1), \quad z_2 = a_1W^{(2)} + b^{(2)}, \quad a_2 = \sigma(z_2)$$

The sigmoid:

- The sigmoid $\sigma(z) = 1/(1 + e^{-z})$ maps the reals into $(0, 1)$. At the output it is read as a probability; in the hidden layer it supplies the nonlinearity.
- Its derivative is written in terms of itself: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, so backpropagation reuses a_1 directly.
- For large negative z , e^{-z} overflows. Computing $e^z/(1 + e^z)$ when $z < 0$ avoids it (a requirement of Task 1).

Why the hidden layer is needed:

- Drop the nonlinearity and stacking layers gives $XW^{(1)}W^{(2)}$, which is one linear model again. The σ in between is what widens the set of functions the extra layer can reach.
- The passing region of this data is a rectangle, and one straight line cannot separate its inside from its outside (Figure 2, left). One hidden unit makes one soft half-plane, and the output unit overlaps those half-planes into a region close to the rectangle (Figure 2, right).

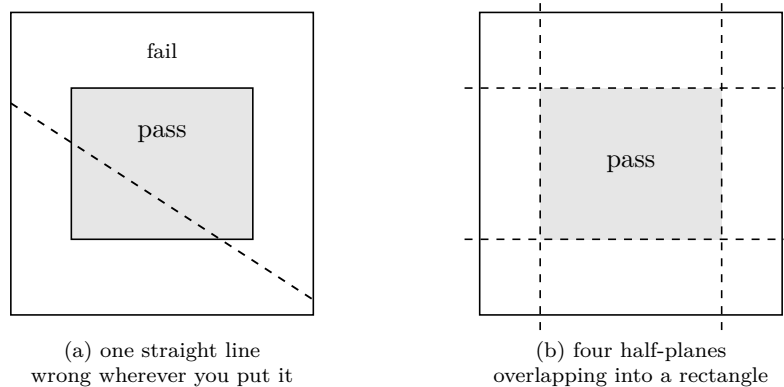


Figure 2. The passing region and the shape of the boundary. Temperature runs along the horizontal axis and pressure up the vertical one; the shaded area is the passing region. The dashed lines are the boundaries each model can draw.

Assumptions and limits:

- The sigmoid's derivative approaches 0 as $|z|$ grows. In deeper networks that is where gradients vanish. From Lab 4-2 on we use the ReLU family.
- Five percent of the labels in this data are flipped, so even a perfectly learned rule caps accuracy near 0.95. If your validation accuracy comes out near 1.0, suspect a leak and check the split first.

3.3 Backpropagation: collecting gradients with the chain rule

Backpropagation starts at the loss and passes partial derivatives back along the forward pass (Figure 3). Multiplying the two output-layer terms cancels $a_2(1 - a_2)$ and leaves something very simple.

$$\frac{\partial L}{\partial a_2} = -\frac{1}{n} \left(\frac{y}{a_2} - \frac{1-y}{1-a_2} \right), \quad \frac{\partial a_2}{\partial z_2} = a_2(1-a_2) \implies \delta_2 = \frac{\partial L}{\partial z_2} = \frac{a_2 - y}{n}$$

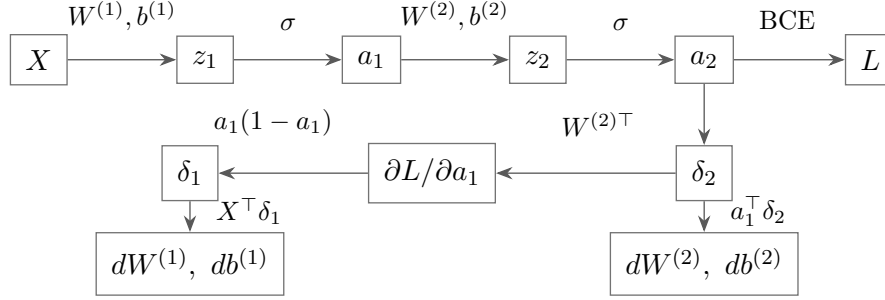


Figure 3. Forward and backward, term by term. The upper arrows are the forward pass, the lower ones backpropagation. Each δ is the partial derivative of the loss with respect to one pre-activation, and a weight gradient is that δ times the output of the layer before it.

- **Weights and biases:** for any layer, the weight gradient is the transpose of that layer's input times δ , and the bias gradient is δ summed down the rows.
- **Passing it back:** $\partial L / \partial a_1 = \delta_2 W^{(2)\top}$ and $\delta_1 = (\delta_2 W^{(2)\top}) \odot a_1(1 - a_1)$, where $\odot$ is the elementwise product.

Spec (Task 2), with n the number of rows in the batch

```
dz2 = (a2 - y) / n, dW2 = a1.T @ dz2, db2 = dz2.sum(axis=0, keepdims=True)
```

```
da1 = dz2 @ W2.T, dz1 = da1 * a1 * (1 - a1)
```

```
dW1 = X.T @ dz1, db1 = dz1.sum(axis=0, keepdims=True)
```

The $1/n$ belongs in $dz2$ and appears exactly once.

Assumptions and limits:

- The derivation holds only for a sigmoid output with binary cross-entropy. Change either one and δ_2 takes a different form.
- Leaving out the $1/n$ keeps the direction right, so training still runs. It multiplies the effective learning rate by n and changes every conclusion in Task 4. The check in the next section catches it.

3.4 Checking the gradient with finite differences

The definition of the derivative gives an approximate gradient from the loss alone. Move one entry of one parameter by ε in each direction and look at the difference in loss.

$$g_{\text{num}} = \frac{L(\theta_i + \varepsilon) - L(\theta_i - \varepsilon)}{2\varepsilon}, \quad \text{relative error} = \frac{|g_{\text{ana}} - g_{\text{num}}|}{\max(10^{-8}, |g_{\text{ana}}| + |g_{\text{num}}|)}$$

Why it is done this way:

- **Central differences:** a one-sided difference has error proportional to ε , while looking at both sides gives error proportional to ε^2 .
- **Relative error:** gradients differ in size from layer to layer, so an absolute error cannot give one threshold. The 10^{-8} in the denominator keeps a near-zero gradient from dividing by zero.
- **One entry at a time:** restore the entry before moving to the next one, or every later comparison is computed at different parameters.

The threshold and its limits:

- A relative error above 10^{-6} means the analytic gradient is wrong. Check the sign, the $1/n$, the transposes, and where the elementwise product sits, in that order.
- Shrinking ε without limit makes things worse, because the rounding error of the loss, around 10^{-16} , is divided by 2ε . The compromise is $\varepsilon = 10^{-5}$, the skeleton default. A passing check does not always reach 10^{-13} : the $W^{(1)}$ entries, whose gradients are smallest, climb to about 10^{-7} , and that is the value `results.json` records.
- A passing check says the gradient is consistent with the loss function. If the loss function itself is wrong, the check passes along with it.

3.5 Learning rate, mini-batches, epochs

An update uses part of the data rather than all of it. That part is a mini-batch, and one sweep through the data is an epoch. This lab splits 640 training rows into batches of 32, so there are 20 updates per epoch and 8000 over 400 epochs.

- For the same amount of computation you get far more updates than one full-data step would give, and the noise from batch-to-batch differences in the gradient helps escape shallow spots.
- The order is reshuffled every epoch. Without reshuffling the same rows recur in the same groups, which puts a periodic bias into the updates.

Three regimes:

Table 1 is a *different, smaller* problem, run through the same code you are about to write: 300 rows of one feature, positive inside a band, 5% of the labels flipped, a 1-8-1 network, 150 epochs of batch size 20. The split, the seed, the initial weights, the epochs, and the batch size are identical across the three rows; only the learning rate differs. Task 4 asks the same question of the lab's data with its own three rates, and you read your own numbers off `results.json`.

Table 1. Three learning rates on a toy problem. Not the lab's data: 300 rows, one feature, 60 validation rows whose majority-class accuracy is 0.6333. An accuracy of 0.6333 therefore means nothing was learned.

learning rate	final train loss	max train loss	final val loss	final val accuracy
0.01	0.6480	0.6837	0.6558	0.6333
0.5	0.2723	0.6608	0.2732	0.9667
50.0	0.6564	14.4410	0.6597	0.6333

- **0.01, too small:** after the full 150 epochs the loss has barely moved from its starting value. The steps point the right way, and many more epochs would get there, but inside this budget the run fails and the accuracy is the majority class.
- **0.5, usable:** the loss falls to 0.27 and validation accuracy reaches 0.9667, more than 30 points above the majority class.
- **50.0, too large:** the training loss reaches 14.4 at one point. Yet the *final* training loss is 0.6564, which looks almost ordinary next to the 0.6480 of the too-small run, because it is one arbitrary point on an oscillating curve. That is why `max_train_loss` is recorded alongside it. On a diverging run the final losses also differ from one machine to another, since a chaotic trajectory magnifies rounding; the maximum is the stable number.
- Validation loss sitting above training loss is common on its own and is not yet overfitting. What makes it overfitting is a validation loss that bottoms out mid-run and then climbs while the training loss keeps falling. Look for that shape in your own curves.

Assumptions and limits:

- A “good learning rate” is good only for one initialization, one batch size, and one architecture. Three rates spread over several orders of magnitude are three points, not the edges of a band. Sweeping the toy problem more finely puts validation accuracy more than 5 points above the majority class from 0.5 up to 10; smaller and larger rates fall back to the majority class. Three points locate a band without measuring it.
- One run is one measurement. Change the seed and the numbers move. Writing that limitation into your report scores better than hiding it.

4 Worked example: one step on a two-row network

Work one forward pass, one backward pass, and one update with the setup in Table 2. There is 1 input, 1 hidden unit, and 1 output, and the batch has 2 rows. Values are printed to four decimals but carried forward unrounded, so redoing the arithmetic from the printed digits can shift the last place by one.

Table 2. The setup of the worked example. The input is the scaled temperature and the target is whether the part passed. The four parameters were chosen arbitrarily. With $n = 2$ you can see where the $1/n$ goes.

	x	y	parameter	value
row 1	0.4	1	$W^{(1)}, b^{(1)}$	1.5, -0.2
row 2	-0.8	0	$W^{(2)}, b^{(2)}$	2.0, 0.5

Step 1, forward pass and loss:

1. $z_1 = [0.4 \cdot 1.5 - 0.2, -0.8 \cdot 1.5 - 0.2] = [0.4, -1.4], \quad a_1 = \sigma(z_1) = [0.5987, 0.1978]$
2. $z_2 = [0.5987 \cdot 2 + 0.5, 0.1978 \cdot 2 + 0.5] = [1.6974, 0.8956], \quad a_2 = [0.8452, 0.7101]$
3. Row 1 has $y = 1$, giving $-\log 0.8452 = 0.1682$; row 2 has $y = 0$, giving $-\log(1 - 0.7101) = 1.2381$

4. $L = (0.1682 + 1.2381)/2 = \mathbf{0.7031}$

Step 2, backward pass:

1. $\delta_2 = (a_2 - y)/2 = [(0.8452 - 1)/2, (0.7101 - 0)/2] = [-0.0774, 0.3550]$
2. $dW^{(2)} = 0.5987 \cdot (-0.0774) + 0.1978 \cdot 0.3550 = \mathbf{0.0239}$, $db^{(2)} = -0.0774 + 0.3550 = \mathbf{0.2776}$
3. $\partial L / \partial a_1 = \delta_2 \cdot 2.0 = [-0.1548, 0.7101]$ and $a_1(1 - a_1) = [0.2403, 0.1587]$, so $\delta_1 = [-0.0372, 0.1127]$
4. $dW^{(1)} = 0.4 \cdot (-0.0372) + (-0.8) \cdot 0.1127 = \mathbf{-0.1050}$, $db^{(1)} = -0.0372 + 0.1127 = \mathbf{0.0755}$

Step 3, update and check ($\eta = 1$):

1. $W^{(1)} = 1.5 - (-0.1050) = 1.6050$, $b^{(1)} = -0.2 - 0.0755 = -0.2755$
2. $W^{(2)} = 2.0 - 0.0239 = 1.9761$, $b^{(2)} = 0.5 - 0.2776 = 0.2224$
3. A second forward pass with the new parameters gives $a_2 = [0.8005, 0.6378]$ and a new loss of $L = \mathbf{0.6190}$, down 0.0841 in one step.

Step 4, the gradient check at the same point:

- Against central differences with $\varepsilon = 10^{-5}$, all four relative errors come out below 10^{-9} , the largest being 1.0×10^{-10} for $b^{(1)}$. All are far below the 10^{-6} threshold.
- Leave out the $1/n$ and $dW^{(1)}$ becomes -0.2100 , exactly twice the numerical value -0.1050 , for a relative error of 0.3333. Without the check this is hard to notice, since training still reduces the loss.
- Where this note and the spec appear to disagree, **the spec is what counts**.

5 Common mistakes

1. **Shapes that broadcast instead of failing.** Store y as $(n,)$ rather than $(n, 1)$ and $a_2 - y$ becomes (n, n) . No exception is raised and only the loss value is wrong. Check that every gradient has the shape of the parameter it belongs to.
2. **The $1/n$ applied twice, or not at all.** The spec puts it in `dz2` once. Averaging again in `db2` divides twice. The relative error then comes out at exactly $(n-1)/(n+1)$, so it approaches 1 as the batch grows: 0.88 on a 16-row check, 0.95 on a 40-row one. Suspect this first whenever the check reports a number near 1.
3. **Discarding the forward intermediates and recomputing them.** If the parameters have already been updated when you recompute a_1 , you get a different value. Pass the cache `forward` returns straight through.
4. **Updating on validation data.** A backward pass while computing the validation loss puts the validation set into the parameters. A hidden test corrupts the validation labels and checks the parameters do not move.
5. **Writing an answer you read off the screen into the code.** The hidden tests run on different data, so a value copied from this dataset is wrong there. Hard-coding is handled as academic misconduct.

6 How this maps to the lab tasks

Table 3. Tasks matched to concept sections. Which section each task draws on, and what grading looks at. Worth checking once before you write the report.

Lab task	Concept	What grading looks at
Task 1. Forward pass	§3.2	shapes, a numerically stable sigmoid, a mean loss
Task 2. Gradients and check	§3.3, §3.4, §4	the chain rule expanded correctly, where the $1/n$ sits, restoring entries
Task 3. Training loop	§3.5	reshuffling each epoch, the update rule, validation kept out
Task 4. Learning-rate comparison	§3.1, §3.5	one factor varies, everything else fixed
Report figures 1–3	§3.2, §3.5	the shape of the passing region, the three curves, the decision boundary

7 Self-check

1. The training loss sits near 0.69 for all 400 epochs. Name three possible causes and say in what order you would check them.
2. Deriving $\delta_2 = (a_2 - y)/n$ makes $a_2(1 - a_2)$ disappear. Where does it cancel, and does the property survive a change of output activation?
3. What happens to the relative error if ε goes from 10^{-5} to 10^{-9} in the central difference? Explain in two sentences.
4. In Table 1 the diverging run ends at almost the same training loss as the too-small run. Which column still lets you call it a failure, and why is that column the trustworthy one?
5. Remove the hidden layer and wire the inputs straight to the output. What model is that, and roughly what validation accuracy would you expect on the lab's data, given the shape of the passing region?

These questions are fair game for the oral check. You must be able to explain every line you submit.

8 Further reading

- Nielsen, *Neural Networks and Deep Learning*, chapter 2, backpropagation
<http://neuralnetworksanddeeplearning.com/chap2.html>
- Goodfellow, Bengio, Courville, *Deep Learning*, sections 6.5 and 8.3
<https://www.deeplearningbook.org/>
- Rumelhart, Hinton & Williams, *Learning representations by back-propagating errors*, Nature 323, 1986
<https://www.nature.com/articles/323533a0>